Policy-Based Profiles for Network Intrusion Response Systems

Kieran Hughes, Kieran McLaughlin, Sakir Sezer
Centre for Secure Information Technologies
Queen's University Belfast
{khughes44, kieran.mclaughlin, s.sezer}@qub.ac.uk

2022 IEEE International Conference on Cyber Security and Resilience (CSR) | 978-1-6654-9952-1/22/$31.00 ©2022 IEEE | DOI: 10.1109/CSR54599.2022.9850304

Abstract— **Attacks on computer networks are increasingly common, often leading to severe economical and reputational damage to organisations. Subsequently, Intrusion Response Systems are recently an active area of research which seek to automatically respond to alerts generated by Intrusion Detection Systems. Current Intrusion Response Systems often seek to find optimal responses based on a general and balanced policy such as the cost and benefit to the network overall. However, organisations are encouraged to prepare Incident Response Policies, which outline prioritisations and performance measures for their response. These policies are highly individualised to the organisation, often influenced by the type of data present within the network. Building on this it is possible for several subsections of a network to have differing Incident Response Policies, for example in a Cyber-Physical network, a Control Area Network may have a much stricter policy in order to preserve a physical process. In this work we utilise a Deep Reinforcement Learning approach to allow the customisation of Reward Functions which in turn facilitates the creation of Response Profiles to align with differing Incident Response Policies. Evaluation of the Profiles is performed in a Cyber-Physical System testbed consisting of Web and Business local area networks configured using Mininet and integrated with a Tennessee Eastman Process plant running in Matlab. Experimentation demonstrates the ability of a Reinforcement Learning Agent to converge on near-optimal response to multi-stage attack scenarios in accordance with their Response Profile.**

Keywords—intrusion, response, systems, multi-profile, response policy

I. INTRODUCTION

High profile cyber-attacks are continuing to cause significant damage to global business and data breaches are becoming increasingly costly. Accordingly, there has been a lot of activity in the research area of Intrusion Detection Systems (IDS), which are tools that automatically identify malicious or suspicious activity on a monitored network. While IDS research has seen significant advancements, which has led to fast detections by utilising machine learning approaches [1], the ability to respond to these alerts in a timely and competent manner is a separate and somewhat juvenile area of research. Intrusion Response can be defined as the sequence of reactive actions upon the detection of malicious activity on a computer network. The goal of Intrusion Response can be said to be to disrupt an ongoing exploit, deny the ability of the attacker to propagate and to mitigate and potential harm to the network and its hosts. An Intrusion Response System (IRS) is a tool which seeks to automate the Intrusion Response process. Significant effort in IRS research has gone into enabling the automated discovery of the optimal response to an attack. Prominent approaches to finding this response include Pareto Analysis, Game-Theory, Dynamic Programming and Reinforcement Learning (RL). While these approaches have been proven to perform well, IRS are generally rigid in their policy, with a singular goal of finding a response which provides the most benefit and least cost across the entire network, possibly leading to overly passive or overly aggressive responses.

In this paper we seek to address these issues using a model-free deep reinforcement learning approach in which the learning agent is the Intrusion Response System decision maker, and a Cyber-Physical System (CPS) testbed is the environment in which the agent acts. The response policy of an agent, which defines the actions it takes in a given situation, can be controlled through the development of a reward function. This allows the creation of unique profiles, tailored to how the agent is expected to react in its environment. We train three profiles; Aggressive, Balanced and Remediation. Results demonstrate that each response profile is successfully able to converge to a near optimal policy. In separate experiments, results show the balanced profile successfully stopping an attack with the minimal cost to the network, the aggressive profile successfully stopping an attack at the first stage and a remediation profile successfully restoring a Tennessee Eastman Process (TEP) to normal operation.

The rest of the paper is presented as follows. Section II outlines background information and discusses closely related work. Section III discusses the use of multiple profiles within an IRS, Section IV defines the methodology of our experiments used to validate the performance of this approach, Section V presents the experimentation results and finally Section VI concludes.

II. BACKGROUND & RELATED WORK

In this section we give an introduction to the three main components which underpin this work, IRS, Security Modelling and Deep Reinforcement Learning. We then provide an overview of the closest related works.

A. Intrusion Response Systems

Intrusion Response Systems are a network-based tool designed to respond to alerts detected by Intrusion Detection Systems. Currently, real-word IRS implementations are generally decision support systems and gain their response capability through macro-like recordings of a previous response [2]. Two immediate problems with these systems are the delayed time to response, and the unlikelihood of identical attacks occurring. When attempting to develop a system which is intelligent enough to automatically deploy responses upon the triggering of an alert, the identification of the optimal response is a complex issue. This complexity has attracted the attention of Security researchers. At a high level, the considerations need to identify an optimal response are illustrated in Figure 1. The illustration of the optimal response in Figure 1 generally follows the balanced response policy taken by the majority of research.

978-1-6654-9952-1/22/$31.00 ©2022 IEEE

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:40:22 UTC from IEEE Xplore. Restrictions apply.



Fig 1. Optimal Response for a Balanced IRS

However, some IRS implementations can have a more specific policy, for example in a network or network segment which contains extremely sensitive data, the response policy may be to stop an attack no matter the cost to network operation, in which case the optimal response may in fact be the aggressive response. This concept can be expanded to cover different policies and we refer to a system which can adapt to multiple policies as a Multi-Profile IRS.

B. Security Modelling

Computational methods to find the optimal response has been the main focus of Intrusion Response research. A key component which facilitates this computation is attack modelling. There are several types of attack models such as Attack Trees, Attack Graphs, Bayesian Attack Graphs and Attack Response Trees. These models are used to represent the potential paths an attacker can take when attempting to reach the resource the defender is attempting to defend, a database for example. The location and type of an alert when mapped to an attack model can provide the defender with a probabilistic method to determining the optimal response. These models can also support more complex approaches such as the formulation of a Markov Decision Process (MDP). Zonouz et al. [3] propose an IRS which includes an automated conversion from an Attack Response Tree to an MDP. The representation of an MDP can be seen in (1).

$$MDP \rightarrow (S, A, P_a, R_a) \quad (1)$$

Where S represents the state space, A represents the action space, P_a represents the probability that taking action a given State S will lead to the next state S', and R_a represents the reward gained from selecting action a given state S. For example, the state space is the possible attack states that the network can be in and the action space is the possible responses available to the IRS given S. Fully observable finite MDPs can be used in stochastic Game Theory [4] or Dynamic Programming to compute state-action policies which provide the greatest long-term reward [5]. Whilst these approaches can be effective in static and small environments, real-world networks are often subject to change. Hence the need to redraw models and recompute optimal policies with every change is problematic. An alternative to the use of static models is the increasingly popular use of machine learning to generalise to unknown information, in particular Deep Reinforcement Learning. Recently, Iannucci et al. [6] has shown that the training of a reinforcement learning agent was able to converge to a near-optimal policy in a non-stationary system.

C. Deep Reinforcement Learning

Reinforcement Learning is an interesting application of machine learning that became popular amongst researchers following Richard Sutton's introductory book in 1992 [7]. The main concept of reinforcement learning is that an agent interacts with an environment and in turn receives a reward based on the goodness of its interaction. The goal of the agent is to maximise the reward it can gain over a period of cycles known as timesteps, the sum of this reward can be defined as (2) where R is the accumulated reward, t is the timesteps and γ represents the discount factor which controls the extent to which the agent favours short-term over long-term reward.

$$R = \sum_{k=0}^{t} \gamma^k r_t + k + 1 \quad (2)$$

However, as the applications of RL increase in size and complexity, it suffers from the state explosion problem, and the storage requirements of the state-actions pairs can become a problem. Deep RL however utilises the generalisation ability of a neural network to estimate a reward when provided an input state and action, removing to need to store large state-action pair tables. While the application and generalisation ability of Deep RL when applied to real world problems is still a research question in itself and requires much work, current leading Policy Gradient algorithms such as Proximal Policy Optimisation have been shown to perform well with minimal tuning [8].

D. Related Work

A recent implementation of multiple response profiles for Intrusion Response is Kholidy's proposal of an IRS for Cyber Physical Systems [9]. Kholidy's proposal, ARC, defines three policies which determine the nodes that are included in the response state space, aggressive, conservative and moderate. With the aggressive policy including more nodes than the conservative policy, hence causing more disruption. The policy is defined by a system administrator. An interesting part of this approach is that the response policy does not directly affect the response applied, instead defining the nodes which are considered during traversal of a Hierarchal Risk Correlation Tree, based on the likeliness of an identified attack traversing though each node. While this in an interesting approach, we seek to investigate the ability of tailored profiles which includes a consideration of the type of response actions applied.

Zolotukhin et al. investigate the performance of two Deep RL algorithms, Deep Q-Network and PPO, when responding to SSH brute-force, distributed denial of service (DDOS) and DNS tunnelling attacks in a software-defined networking (SDN) environment [10]. The authors define a state based on several network attributes such as packet counts and security alerts. This state is then used by the RL agent to determine the changes to the underlying SDN rules. The performance of PPO in this approach is shown to reach a near optimal reward in the shortest number of timesteps, which contributed to our decision to use PPO in our approach. While this work illustrates success at stopping singular attack stages, we seek to investigate the ability of an IRS which can respond to multi-stage attacks.

Iannucci et al. also implement a Deep RL IRS [6]. The authors acknowledge the curse of dimensionality, considering the difficulties around stateful attack modelling. A model-free approach is used to address this problem,

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:40:22 UTC from IEEE Xplore. Restrictions apply.

removing the need to create complex models of the environment, instead allowing the agent to learn the best response in a given state through direct interaction with the environment, essentially a trial-and-error approach. This work indicates the potential for a distributed Deep RL approach which can then generalise to network changes. The authors use a singular Reward Function for their experimentation (3).

$$R(S_t, a, S_{t+1}) = -w_t \frac{T(a)}{T_{max}} - w_c \frac{C(a)}{C_{max}} \quad (3)$$

Where w_c, $w_t \in [0,1]$ represent the weight placed on the execution cost and the execution time, respectively. Reward Function creation is a particularly interesting area of Deep RL. It facilitates the evaluation of state parameters upon the execution of a response. While Iannucci et al. successfully use a simple function, application to real-world systems can benefit from less abstraction when using state parameters. For example, the execution cost *C(a)* can be seen as a derived value, with a number of contributing factors, examples being packet loss, node availability and service availability. These tangible metrics can be observed in the network and passed to the reward function. In our work we seek to evaluate the use of material state parameters to provide more control in the creation of multiple policy-based profiles for RL agents.

Huang et al. use Q-Learning to find the optimal response strategy when a Cyber-Physical System (CPS) is modelled as a two-player stochastic game [11]. Complete mode-based approaches avail of the availability of complete and predefined state information, hence the goal of Q-Learning in this circumstance is to compute the optimal defence strategy based on the entire game state space. As the model is known, the reward is precomputed, the authors refer to this a Payoff and is the negative of the attacker's reward, defined in (4).

$$U_t^A(A_{t,i}, D_{t,i}) = \varepsilon(A_{t,i}, D_{t,i})R(A_{t,i}) + T(D_{t,i}) - C(A_{t,i}) \quad (4)$$

Where $U_t^A(A_{t,i}, D_{t,i})$ is the reward for the unified attack-defence action pair in state t. $C(A_{t,i})$ is the time needed by the attacker to implement the action $A_{t,i}$. $T(D_{t,i})$ is the time needed by the attack to apply the defensive action $D_{t,i}$ and $R(A_{t,i})$ is the time taken to recover a compromised device, i. While it is an interesting approach to evaluate the defender reward gain based on unified attacker and defender metrics, when applied in a real-world environment it may be necessary to consider further metrics than time, for example the time to respond may be fairly low but the negative effect of the response on the network may be high. Another interesting aspect of Huang et al.'s approach is the use of a CPS testbed. In this paper we replicate the use of the Tennessee Eastman Process [12]. However, we seek to integrate the Matlab TEP model with an emulated Mininet network in order to evaluate additional network-based state parameters in our reward functions.

III. MULTI-PROFILE IRS

Through our evaluation of related work, we can appreciate that while current approaches to IRS utilise a range of differing state variables to determine the optimal response, the reward functions which drive the translation of these parameters into a reward value are static. This prevents a decision maker from easily accommodating differing security policies throughout the network. Changes to the reward function of a trained model is not practical, as the agents understanding of how to gain reward is invalidated. Therefore, in this work we demonstrate the application of multiple trained agents, using a variety of differing reward functions. Hence facilitating an IRS with interchangeable profiles unique to the security policy of the network environment in which they are deployed. In Figure 1 we presented different categories of response based on the goals of the defender, we aim to investigate the application of these categories by developing representative reward functions, therefore below we define an Aggressive Policy, Remediation Policy and Balanced Policy. However, before the development of the reward functions, it is necessary to define the state parameters that will be used, as they directly impact the complexity of the reward functions.

A. State Parameter Selection

The National Institute of Standards and Technology (NIST) published a Computer Security Incident Handling guide [13] in which they provide criteria which should be considered when selecting your response policy. In Table 1 we present these criteria and our selected State Parameters which we believe can represent these criteria in a Reward Function. These parameters are values collected from the testbed when the RL agent observes its environment.

TABLE I. SELECTION OF STATE PARAMETERS

NIST Criterion	Selected State Parameters	Parameter Value Ranges
Potential Damage/Theft of Resources	Alert Location	[0,..,*4*]
	Alert Type	[*Mild, Moderate, Severe, Critical*]
	Packet Loss	[0,.., 6]
Need for Evidence Preservation	n/a	n/a
Service Availability	Nodes Offline	[0,..,6]
Time/Resources need to implement Response Policy	Nodes Offline	[0,..,6]
	Packet Loss	[0,.., 6]
Effectiveness of Policy	Alert Location	[0,..,*4*]
	Alert Type	[*Mild, Moderate, Severe, Critical*]
	Responses Applied	[0,..,7]
Duration of Response	Elapsed Time	[0,..,*n*]
Environment Specific	Reactor Pressure	[2600,..,3000]
	Product Quality	[50,..,60]

In addition to the NIST criteria that we use to guide our selection of state parameters, we add the Environment Specific criteria, this is due to the need for Automated IRS to

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:40:22 UTC from IEEE Xplore. Restrictions apply.

acknowledge the importance of significant physical network components. In this work, we use a CPS in which the status of the TEP needs to be closely monitored and controlled, hence requiring the addition of more specific state parameters.

An important observation here is that the approach to Intrusion Response in this paper is for primary response only. While some of NIST's criteria such as the need for evidence preservation may be more applicable to later stages of a response policy, which includes the need for host-based forensics. In this work we are primarily focused on the immediate response upon alert generation by a network-based Intrusion Detection System (IDS). In Table I, Alert type is defined based on the category of attack and in this paper we make the assumption that this is part of the IDS alert, the classification of an alert is set prior to detection, a common procedure in the Snort IDS. For example, port scanning is categorised as a Mild alert, whereas an indicator of compromise (IOC) of a vulnerability exploit is classed as severe. Critical alerts refer to physical system tampering which could cause physical harm. Elapsed time is a unique State Parameter in that it is not used directly in the reward functions however can be used in the termination function, which defines for the RL agent the state in which to terminate.

B. *Reward Functions*

The first profile we implement is the Balanced profile, it seeks to find the best general response overall, with no specific policy-based aims. This is similar to the majority or related work which analyse a general cost and benefit value for each response. The Balanced Profile is defined in (5).

$$R = ((n_{max} - n) + (3.a_{max} - 2.a) + (3.z_{max} - 2.z) + (y_{max} - y) - p)/50 \quad (5)$$

Where R represents the reward gained, n represents the nodes offline, a is alert type, where Mild is represented as 1 and incremented as the severity increases, with Critical being represented as a 4. Alert Location is represented as z, Responses Applied as y. Packets dropped is represented as p where t is the total number of packets dropped and $p = t/100$. The reasoning behind (5) is to include a range of state parameters to ensure a balanced approach whilst still providing a weighting to Attack Location and Attack Type to ensure that the primary goal is to stop the attack.

The second profile we implemented is the Aggressive Profile. The purpose of this profile is to allow the deployment of an IRS which seeks to prioritise the halting of an attack regardless of the response's negative effect on network performance. This profile can be particularly useful in a network or subnet which contains highly sensitive data. A particular use case may be CPS, due to the often critical nature of the physical systems connected to the network. For example, an organisations response policy may be somewhat tolerant in its internet facing subnet while aggressive in its physical control subnet. This flexibility between policies is what we seek to facilitate. The Aggressive Profile is defined in (6).

$$R = \left(\frac{z_{max}}{z} + \frac{a_{max}}{a}\right)/50 \quad (6)$$

In (6) we use only Alert Type and Alert Location to incentivise the RL agent. This allows the agent to find responses which solely effect the progress of the attack, without considering the availability of the network. In turn, applying high-cost responses which also provide the greatest reduction in risk.

The Remediation profile provides us with a more complex task in terms of training a RL agent to restore its network to normal operation. For example, if our CPS is subject to an attack which manipulates the TEP, our agent needs to understand the extent of the manipulation through observation of state parameters while also learning a sequence of response actions which can restore the system to normal operation. While this integration with the operation of a physical system borders a process control problem, the remediation can be applied in a simpler concept, understanding and resolving the occurrence of packet loss due to an attack, for example. The function to evaluate the reactor pressure and product quality readings is defined in (7). Where m is the baseline median reading, σ is the baseline standard deviation and y is the current reading.

$$v = \frac{d^2}{2\sigma} \; where \; d = y - m \quad (7)$$

It is noted that when the RL agent observes the environment, it is a singular call, therefore due to the oscillating nature of sensor readings at any single time of capture, a reading may indeed lie within a normal range, despite the maximum and minimum values reached being strong outliers. In order to address this, the value for y is a maximum value from a buffer of 5 readings. The reward function for the Remediation Policy is then defined in (8), where v_r is v in respect to the Reactor Pressure state parameter, v_p is v in respect to the Product Quality state parameter and k is the number of responses applied.

$$(n_{max} - n) + (p_{max} - p) + w_r\, v_r + w_p v_p - 5\mathrm{k} \quad (8)$$

In (8) we provide a reward based on the number of nodes still operational, the lack of packet errors and the lack of anomalous readings from the TEP. When developing a remediation-based reward function, it may be necessary to use weightings in order to prioritise aspects of the network which are seen as more critical. In the example of our testbed which includes the TEP, weightings are placed on the reactor pressure and product quality with w_r and w_p respectively.

We have presented three reward functions, the training of RL agents with each policy presents the ability of multiple profiles, which in turn can be applied to networks based on the response policy defined by the administrators. While we present specific profiles, the purpose of this work is to illustrate the novel approach to the IRS response selection process, real world implementations should follow the same methodology to implement specific profiles with customised reward functions.

IV. METHODOLODY

The training of Deep Reinforcement Learning agents typically requires a large number of experience episodes. Training also relies on direct interaction with the environment, consequently, it is not practical to train using the real network environment. In this work we utilise a hybrid simulation-emulation approach in which we emulate a network and physical process using Mininet [14] and

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:40:22 UTC from IEEE Xplore. Restrictions apply.

Matlab respectively. We partially simulate attack states, specifically the network-based attack steps. This hybrid approach results in an ~20 seconds episode time. While this is slower than the episode time of fully simulated approaches [6] we believe it provides a more realistic evaluation and importantly allows the evaluation of real state parameters from the Mininet network and the Matlab Tennessee Eastman Process.

A. Testbed

Mininet was chosen as the network emulator for this work due to its rapid prototyping trait and its Python API. Mininet also uses Software-Defined Networking (SDN) which allowed us to create our own controller, in turn facilitating dynamic and fast firewall configuration changes. As shown in Figure 3, the network consists of 6 nodes, within 3 subnets. An outward facing web demilitarised zone (DMZ) containing a web server and associated database, a business LAN containing two computing nodes and a control LAN containing a Historian and Human Machine Interface (HMI). As the network is built and run using a Python API, it can also directly interact with the Tennessee Eastman Process running in Matlab through the Matlab Python API. This integration is controlled by a Python script which also controls the TensorForce Reinforcement Learning agent.

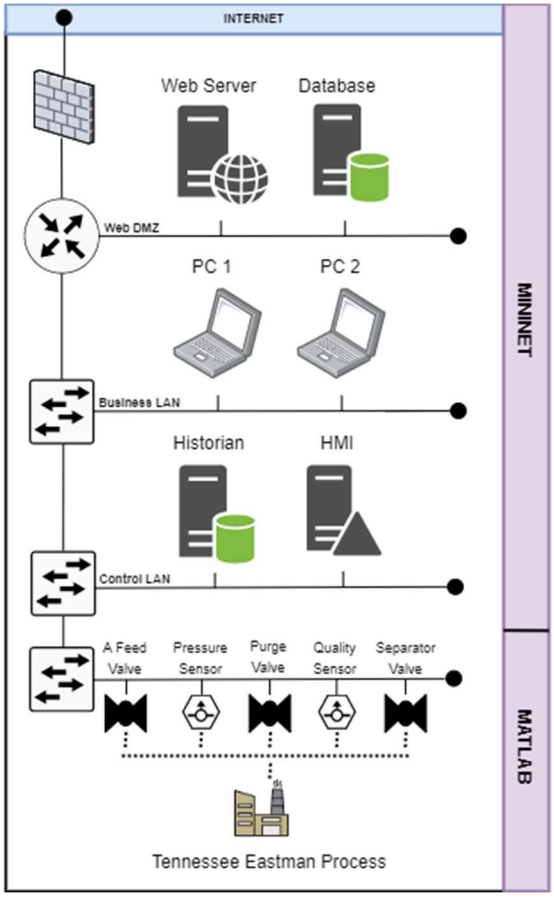


Fig 3. Mininet Network Testbed

The Tennessee Eastman Process is a continuous chemical process which is commonly used in process control research, however it has recently become a popular case study within other fields, including Cyber Security. Components of the process are adjustable such valves which control the rate of inputs and outputs to the reactor. The TEP provides a runtime output which can be monitored by an IRS. In this work we pay particular attention to the Reactor Pressure and Product Quality outputs. Our experimentation will include attacks which target this process. Figure 4 illustrates the reading from the Quality Sensor after suffering an attack at around the 100 hours mark.

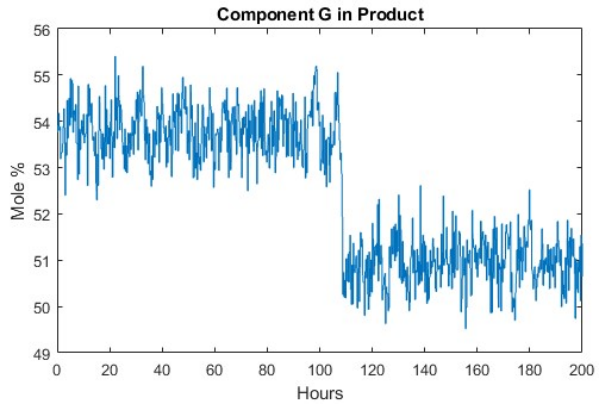


Fig 4. Destabalisation Of Product Quality

B. Attack Scenarios

In this work we investigate the performance of the individual IRS profiles against two attack scenarios. The initial attack is from an external attacker, who first exploits a vulnerability in the web server, pivots through the network and ultimately launches an aggressive attack on the TEP. The second attack scenario is an insider attack which causes significant damage to the network however makes a subtle change to the TEP. The multiple stages of both attacks are illustrated in Figure 5.

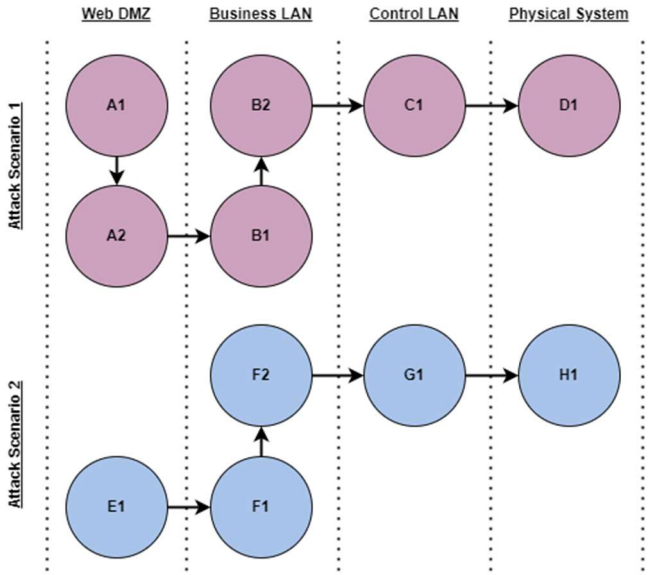


Fig 5. Multi-Stage Attack Scenarios

The description of each attack stage is provided in Table II. Alert Location and Alert Type represent the expected alert information from a non-simulated approach. The location variable refers to the subnet in which the alert is detected, with 0 denoting that the attack has not yet started. 1 being the Web DMZ, 2 representing the Business LAN, 3 representing the Control LAN and 4 representing the TEP. Alert Type is based on the nature of the alert detected and corresponds to the Alert Type Parameter in Table I, with 1 representing

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:40:22 UTC from IEEE Xplore. Restrictions apply.

'Mild' and 4 representing 'Critical'. An obvious observation of the proposed attack scenarios is the sequential nature of the stages. Given the normal state of the network, all stages of the attack successfully complete. Hence ensuring that failure to progress to the next stage of the attack scenario is directly resultant of IRS actions.

TABLE II. ATTACK STAGES

Stage	Description	Alert Location	Alert Type
A0	External Attacker	0	0
A1	Probe Web Server	1	1
A2	Exploit Server Vulnerability	1	3
B1	Probe pivot to Business LAN	2	1
B2	Brute Force Business PC	2	3
C1	Connect to HMI	3	2
D1	Close reactor output valve 5%	4	4
E0	Internal Attacker	0	0
E1	Shutdown Database server	1	3
F1	Scan network	2	1
F2	Disable PC network interface	2	2
G1	Pivot to HMI	3	1
H1	Adjust Mole % G	4	4

C. Response Selection Process

Deep Reinforcement Learning agents learn to implement the best response given their current state in order to maximise the reward gained over a series of timesteps. A collection of timesteps which result in a terminal state is known as an episode. The training process of the RL agent typically consists of a large number of episodes. At the completion of an episode, the total experience gained from interacting with the environment is stored. Upon the completion of a predefined number of episodes, the experience gained is used to update the policy of the RL agent. The number of experience episodes used in this policy update is known as the batch size. The general interaction between the RL agent and an environment is shown in Figure 6, where A_s is the action selected by the Agent given the current state, $R_{s,a}$ is the Reward gained from the environment after taking action *a* in the state *s* and S is the new state.

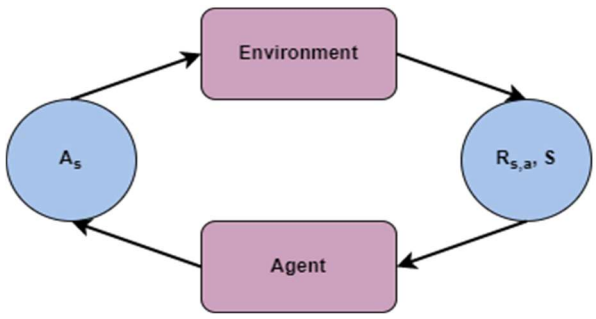


Fig 6. Singular RL Timestep

Hence in this work, the RL Agent assumes the role of the IRS and selects response actions based on the current state of the Environment, which is defined by the State Parameters in Table I. The 26 actions available to the Agent in any given state are defined in Table III. The agent then learns optimal actions based on continuous policy updates. The response selection process and use of state parameters is explained in greater detail in [15].

TABLE III. RESPONSES

Action	Description	Parameter Range
1	Block All Traffic	N/A
2-5	Shutdown Node	Web Server, Database, PC 1, PC 2
6-9	Start Node	Web Server, Database, PC 1, PC 2
10-13	Disable Interface	PC 1, PC 2, Historian, HMI
14-17	Enable Interface	PC 1, PC 2, Historian, HMI
18-21	Block Node	PC 1, PC 2, Historian, HMI
22-25	Adjust TEP Valve	A Feed Valve, Purge Valve, Separator Valve, Mole %
26	Take No Action	N/A

V. EXPERIMENTATION

To evaluate the performance of the unique profiles against the proposed attack scenarios, we train the Aggressive and Balanced profiles on Attack Scenario 1 and the Remediation Profile on Attack Scenario 2. Initially, for the aggressive and balanced reward functions, we evaluated the ability of the RL agent to converge on a response policy which indeed stops the attacks, validating the core functionality of the approach. We then sought to evaluate the actions taken by each profile, in order to illustrate conformity to the type of profile. Finally, we investigated the performance of the remediation profile, specifically its ability to restore the network and physical testbed to normal operation.

A. Aggressive Profile Performance

The training of a RL Agent with an Aggressive profile performed well, the Agent began to learn the actions to gain high reward quite early, with a large increase in reward gained after approximately 120 episodes. The average reward gained per 10 episodes is illustrated in Figure 7.

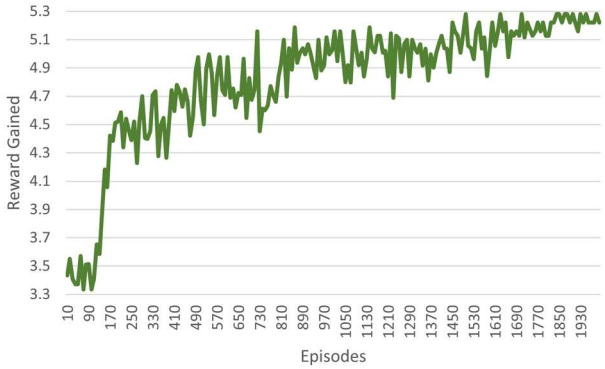


Fig 7. Aggressive Profile Average Reward Gained

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:40:22 UTC from IEEE Xplore. Restrictions apply.

In order to validate that this reward corresponds to an effective response from an IRS perspective, we monitored the stage to which the attack scenario reached. The average stage reached per 10 episodes is illustrated in Figure 8.

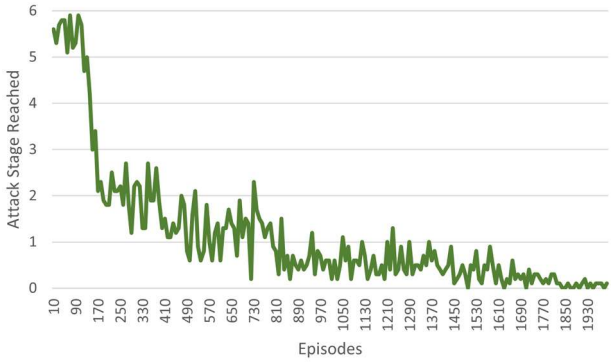


Fig 8. Aggressive Profile Average Attack Stage Reached

B. Balanced Profile Performance

The Balanced profile utilises a larger number of state parameters in order to ensure a good view of the overall health of the network, therefore as expected the agent improved at a much slower rate. The average reward gained per 10 episodes is illustrated in Figure 9.

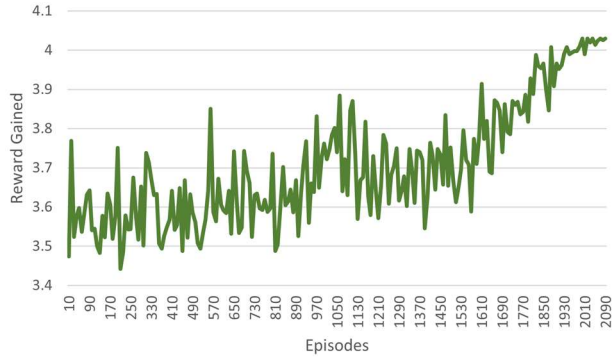


Fig 9. Balanced Profile Average Reward Gained

Compared to the Aggressive profile, this agent was required to consider how its actions effected the availability of the network despite the prioritised goal of stopping the attack.

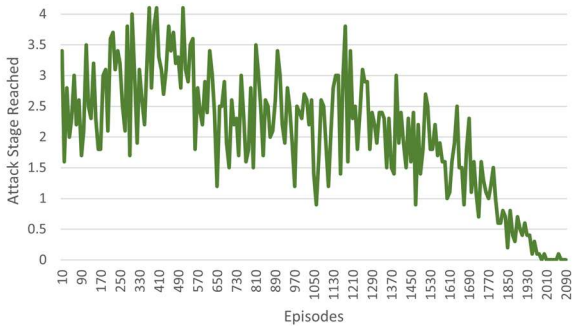


Fig 10. Balanced Profile Average Attack Stage Reached

The average attack stage reached per 10 episodes is illustrated in Figure 10.

C. Balanced vs Aggressive Action Selection

While the reward gain is a good indicator of the performance of the RL agent, we analyse the actions taken for a number of reasons including validation. Figure 11 illustrates the frequency of each action progressively from the start of training, for the Aggressive Profile. Specifically, the frequency per 500 total actions.

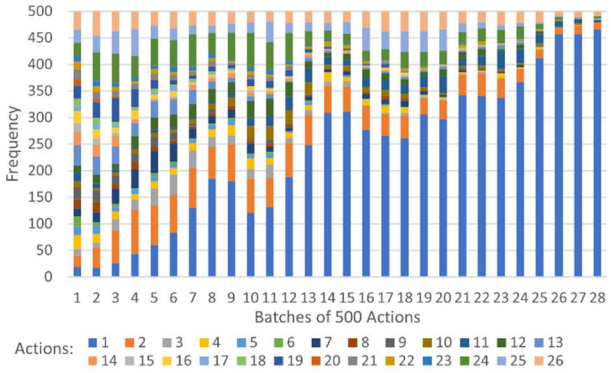


Fig 11. Action Frequency Per 500 Actions, Aggressive Profile

Based on Figure 11 we can see that the RL agent started with random action selection as expected, it then began to favour certain actions as its exploration decreased. The blue bar which we can see becomes increasingly frequent as the training progresses represents Action 0, which updates the firewall rules to block all traffic in the network. Understandably this successfully stopped the progression of the attack, but at a high cost to network operation.

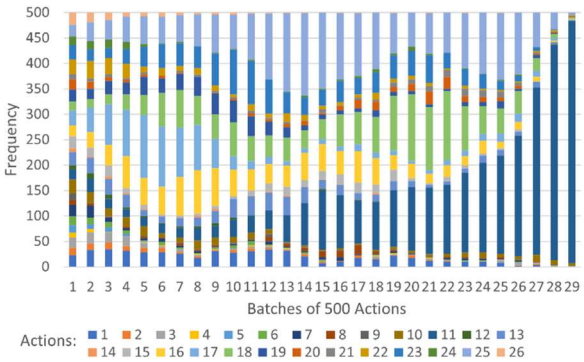


Fig 12. Action Frequency Per 500 Actions, Balanced Profile

Figure 12 shows the RL agent favoring a number of actions as the training progressed. Actions 25 and 17, represented by the light blue and light green respectively were highly favored during the training. Action 17 added a firewall rule to block traffic from the Database and Action 25 was the take no Action response. Blocking traffic from the Database provided no benefit in stopping the attack, while taking no action provided a small reward. Unsurprisingly the RL Agent then began to massively favor Action 10, represented by the dark blue, as it disables a network interface on the Web Server which stops the attack at the first stage. While this is a near optimal response based on a balanced profile, the Agent may have acquired more reward though the initial selection of Action 10 followed by Action 25 for the remaining timesteps.

D. Remediation Profile Performance

A Remediation-based profile is less focused on stopping an attack, its goal is to restore the network to normal operation during or post attack. In order to evaluate the performance of this profile, we use Attack Scenario 2, in which an insider attack causes random harm to all aspects of

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:40:22 UTC from IEEE Xplore. Restrictions apply.

the network, including the TEP. Figure 13 illustrates the average reward gained per 10 training episodes.



Fig 13. Remediation Profile Average Reward Gained

The RL agent using the Remediation focused reward function learned well and was able to converge to a high reward after ~500 episodes. However, the agent identified the need to restore the TEP and prioritised it over the rest of the network, the recovery is illustrated in Figure 15.

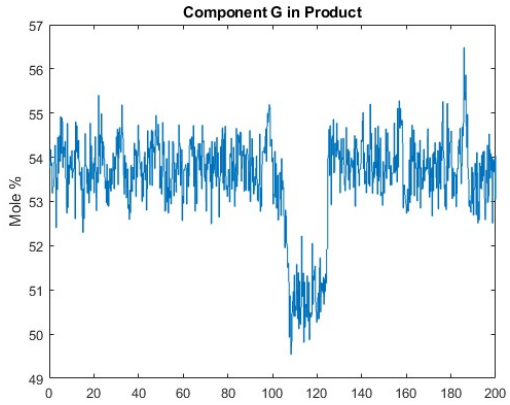


Fig 15. TEP Attack Recovery

VI. Conclusion & Future Work

In this paper we have demonstrated the ability of a Deep Reinforcement Learning based IRS to adopt differing Response Profiles using Reward Function customisation. This facilitates the deployment of an IRS which has more flexibility to align with the unique Incident Response Policy of an organisation. Experiments show the RL agents successfully stop attacks using the Aggressive and Balanced profiles. An agent configured with a Remediation profile demonstrated the ability to restore a physical process to normal operation. The proposed approach builds on [9] through the coupling of the Response Profile and actions taken. We also build on [11] through the combination of state parameters from both the TEP and the Mininet network.

The main contribution of this work is the unique use of reward function design to demonstrate a closer alignment to custom network incident response policies, in the related work section we discuss reward functions used by other reinforcement learning proposals and their goals/limitations. It is the nature of IRS proposals, and in particular experimentation on conceptual components of an IRS, to lack comparison criteria. For example, in the closest related work [6] the only point of qualitative comparison is the rate of convergence however that comparison provides no beneficial justification to the outcomes of this paper.

As future work we seek to evaluate the use of multiple profiles within a singular network, for example the use of an Aggressive agent which prioritises stopping the attack alongside a Remediation agent. A limitation of Deep Reinforcement Learning in this approach is the difficulty in generalisation, therefore we seek to investigate a distributed approach which can result in a smaller impact from change to the network or attack.

References

[1] Z. Ahmad, A. Shahid Khan, C. Wai Shiang, J. Abdullah, and F. Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," *Trans. Emerg. Telecommun. Technol.*, vol. 32, no. 1, pp. 1–29, 2021, doi: 10.1002/ett.4150.

[2] Cynet, "Cynet - Response Orchestration," *Response Orchestration*. https://www.cynet.com/platform/response/response-orchestration/ (accessed Oct. 14, 2019).

[3] S. A. Zonouz, H. Khurana, W. H. Sanders, and T. M. Yardley, "RRE: A game-theoretic intrusion response and recovery engine," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 2, pp. 395–406, 2014

[4] B. Wang, Y. Sun, M. Sun, and X. Xu, "Game-theoretic Actor-critic based Intrusion Response Scheme (GTAC-IRS) for wireless SDN based IoT Networks," *IEEE Internet Things J.*, vol. X, no. 8, pp. 1–1, 2020

[5] R. Bellman, "Dynamic programming," *Math. Sci. Eng.*, vol. 40, no. P1, pp. 101–137, 1967

[6] S. Iannucci, V. Cardellini, O. D. Barba, and I. Banicescu, "A hybrid model-free approach for the near-optimal intrusion response control of non-stationary systems," *Futur. Gener. Comput. Syst.*, vol. 109, pp. 111–124, 2020

[7] R. Sutton and A. Barto, "REINFORCEMENT LEARNING: AN INTRODUCTION," *Robotica*, vol. 17, no. 2, pp. 229–235, 1999

[8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," pp. 1–12, 2017, [Online]. Available: http://arxiv.org/abs/1707.06347.

[9] H. A. Kholidy, "Autonomous mitigation of cyber risks in the Cyber–Physical Systems," *Futur. Gener. Comput. Syst.*, vol. 115, pp. 171–187, 2021

[10] M. Zolotukhin, S. Kumar, and T. Hamalainen, "Reinforcement Learning for Attack Mitigation in SDN-enabled Networks," in *2020 6th IEEE International Conference on Network Softwarization (NetSoft)*, 2020

[11] S. Huang, C. Zhou, N. Xiong, S. H. Yang, Y. Qin, and Q. Zhang, "A general real-time control approach of intrusion response for industrial automation systems," *IEEE Trans. Syst. Man, Cybern. Syst.*, vol. 46, no. 8, pp. 1021–1035, 2016, doi: 10.1109/TSMC.2015.2469688.

[12] N. L. Ricker, "Decentralized control of the Tennessee Eastman Challenge Process," *J. Process Control*, vol. 6, no. 4, pp. 205–221, 1996

[13] K. S. Paul Cichonski, Tom Millar, Tim Grance, "Computer Security Incident Handling Guide : Recommendations of the National Institute of Standards and Technology," *NIST Spec. Publ.*, vol. 800–61, p. 79, 2012, [Online]. Available: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf.

[14] A. Mayssara A. Abo Hassanin Supervised, *International Conference on communication, computing & systems*. 2014.

[15] K. Hughes, K. McLaughlin, and S. Sezer, "A Model-Free Approach to Intrusion Response Systems," *J. Inf. Secur. Appl.*, vol. 66, no. March, p. 103150, 2022, doi: 10.1016/j.jisa.2022.103150.

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:40:22 UTC from IEEE Xplore. Restrictions apply.